```
// #1 blinking led
import RPi.GPIO as x
import time
x.setwarnings(False)
x.setmode(x.BOARD)
x.setup(7, x.OUT)
x.setup(11, x.OUT)
x.setup(13, x.OUT)
x.setup(15, x.OUT)
while True:
for i in range(3):
x.output((7,11,13,15), (1,1,1,1))
time.sleep(0.5)
x.output((7,11,13,15), (0,0,0,0))
time.sleep(0.5)
for i in range(3):
x.output((7,11,13,15), (1,1,0,0))
time.sleep(0.5)
x.output((7,11,13,15), (0,0,1,1))
time.sleep(0.5)
for i in range(3):
x.output((7,11,13,15), (1,0,0,0))
time.sleep(0.5)
x.output((7,11,13,15), (1,1,0,0))
time.sleep(0.5)
x.output((7,11,13,15), (1,1,1,0))
time.sleep(0.5)
x.output((7,11,13,15), (1,1,1,1))
time.sleep(0.5)
x.output((7,11,13,15), (1,1,1,0))
time.sleep(0.5)
x.output((7,11,13,15), (1,1,0,0))
time.sleep(0.5)
x.output((7,11,13,15), (1,0,0,0))
time.sleep(0.5)
x.output((7,11,13,15), (0,0,0,0))
time.sleep(0.5)

//3  4 digit segment
import time
import datetime
import tm1637
Display = tm1637.TM1637(clk=23, dio=24)
print("4 Digit Display - Digital Clock")
# Start with colon ON
colon = True
while True:
now = datetime.datetime.now()
hour = now.hour
minute = now.minute
Display.numbers(hour, minute, sep=colon)
colon = not colon
time.sleep(1)
```

```
//5 ads1115 oscilloscope
#enable i2c sudor raspi-config
#install sudo apt-get install build-essential python3-dev python3-smbus git
#change its folder  git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git
cd Adafruit_Python_ADS1x15
sudo python3 setup.py install
import time
import board
import busio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)
chan = AnalogIn(ads, ADS.P0)
print("{:>5}\t{:>5}".format("raw", "v"))
while True:
  print("{:>5}\t{:>5.3f}".format(chan.value, chan.voltage))
  time.sleep(0.5)
#after running code sudo apt-get install python3-matplotlib
sudo apt-get install python3-pip
sudo pip3 install drawnow
#to run the graph git clone https://github.com/tyitraspberrypi/Adafruit_Python_ADS1x1z_graph.git
import time
import board
import busio
import adafruit_ads1x15.ads1015 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
import matplotlib.pyplot as plt
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1015(i2c)
chan = AnalogIn(ads, ADS.P0)
times = []
voltages = []
start_time = time.time()
plt.ion()
fig, ax = plt.subplots()
while True:
  current_time = time.time() - start_time
  voltage = chan.voltage
  times.append(current_time)
  voltages.append(voltage)
  ax.clear()
  ax.plot(times, voltages, label="Voltage (V)")
  ax.set_xlabel("Time (s)")
  ax.set_ylabel("Voltage (V)")
  ax.set_title("ADS1015 Voltage Readings")
  ax.legend()
  plt.pause(0.1)
  time.sleep(0.5)


#4 gps
sudo systemctl stop serial-getty@ttyS0.service
sudo systemctl disable serial-getty@ttyS0.service
sudo reboot
sudo systemctl enable serial-getty@ttyAMA0.service
sudo apt-get install minicom
sudo cat /dev/ttyAMA0
import time
import RPi.GPIO as GPIO
import pio
```

```
import Ports
GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
pio.uart=Ports.UART () # Define serial port
while 1:
    Data=pio.uart.recv()
    pio.uart.print(Data)


#6 16x2 Lcd
#enable i2c sudo raspi-config
sudo apt update
sudo apt install i2c-tools python3-smbus python3-pip
pip3 install RPLCD
i2cdetect -y 1
from RPLCD.i2c import CharLCD
 from time import sleep
lcd = CharLCD('PCF8574', 0x27, cols=16, rows=2)
lcd.write_string("Hello from Pi 400!")
sleep(3)
 lcd.clear()


#7 RFID module to read/write tags/cars
#sudo raspi-config     interfacing option->serial
lsmod | grep spi
sudo pip3 install mfrc522
#write data
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522
reader = SimpleMFRC522()
try:
text = input (“Enter New data:”)
print('Place tag')
reader.write (text)
print ("Data written successfully!")
finally:
GPIO. cleanup()
#read
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522
reader = SimpleMFRC522()
try:
print("Place the RFID tag near the reader...")
id, text = reader.read()
print("Tag ID:" ,id)
print("Data stored:" ,text)
finally:
GPIO.cleanup()
#To erase data
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522
reader = SimpleMFRC522()
try:
print("Hold your RFID tag near the reader to erase/delete data...")
# Overwrite the tag data block with empty text (blank spaces)
reader.write(" ")
print("SUCCESS: Tag data wiped successfully!")
except Exception as e:
print(f"An error occurred: {e}")
finally:
```

```
GPIO.cleanup()


#8 fingerprint sensor
sudo raspi-config  (Enable serial port)
pip3 install adafruit-circuitpython-fingerprint
ls/dev/ttyUSB*
import time
import board
import adafruit_fingerprint
import serial
uart = serial.Serial("/dev/ttyUSB0", baudrate=57600)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
def get_fingerprint():
  print("Waiting for image...")
  while finger.get_image() != adafruit_fingerprint.OK:
    pass
  print("Templating...")
  if finger.image_2_tz(1) != adafruit_fingerprint.OK:
    return False
  print("Searching...")
  if finger.finger_search() != adafruit_fingerprint.OK:
    return False
  return True
def enroll_finger(location):
  for fingerimg in range(1, 3):
    if fingerimg == 1:
      print("Place finger on sensor...", end="")
    else:
      print("Place same finger again...", end="")
    while True:
      i = finger.get_image()
      if i == adafruit_fingerprint.OK:
        print("Image taken")
        break
      elif i == adafruit_fingerprint.NOFINGER:
        print(".", end="")
      else:
        print("Other error")
        return False
    print("Templating...", end="")
    i = finger.image_2_tz(fingerimg)
    if i == adafruit_fingerprint.OK:
      print("Templated")
    else:
      print("Other error")
      return False
    if fingerimg == 1:
      print("Remove finger")
      time.sleep(1)
      while i != adafruit_fingerprint.NOFINGER:
        i = finger.get_image()
  print("Creating model...", end="")
  i = finger.create_model()
  if i == adafruit_fingerprint.OK:
    print("Created")
  else:
    print("Error")
    return False
  print(f"Storing model #{location}...", end="")
  i = finger.store_model(location)
```

```
  if i == adafruit_fingerprint.OK:
    print("Stored")
  else:
    print("Error!")
    return False
  return True
def get_num():
  i = 0
  while (i > 127) or (i < 1):
    try:
      i = int(input("Enter ID # from 1-127: "))
    except ValueError:
      pass
  return i
while True:
  if finger.read_templates() != adafruit_fingerprint.OK:
    raise RuntimeError("Failed to read templates")
  print("Fingerprint templates:", finger.templates)
  print("e - To enroll")
  print("f - To find")
  print("d - To delete")
  c = input("> ")
   if c == "e":
    enroll_finger(get_num())
    print("Stored successfully!")
  if c == "f":
    if get_fingerprint():
      print("Detected #", finger.finger_id)
    else:
      print("Finger not found")
  if c == "d":
    if finger.delete_model(get_num()) == adafruit_fingerprint.OK:
      print("Deleted!")
    else:
      print("Failed to delete")


#11 fingerprint % lcd
#sudo raspi-config (enable i2c , serial port hardware serial yes
sudo apt-get update && sudo apt-get install -y i2c-tools python3-pip python3-serial
pip3 install RPLCD adafruit-circuitpython-fingerprint smbus2
#enroll.py
import time, serial, adafruit_fingerprint
uart = serial.Serial("/dev/serialo", baudrate=57600,
timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint (uart)
location = int(input("Enter Slot ID (1-127): "))
print("Place finger on sensor ... ")
while finger.get_image() != adafruit_fingerprint.OK:
        pass
finger.image_2_tz(1)
print ("Remove finger ... ")
time.sleep(2)
while finger.get_image()==adafruit_fingerprint.OK:
        pass
print("Place SAME finger again ... ")
while finger.get_image()!=adafruit_fingerprint.OK:
        pass
finger . image_2_tz(2)
```

```
if finger.create_model() == adafruit_fingerprint.OK:
        if finger.store_model(location)==adafruit_fingerprint.OK:
                print(f"Enrolled successfully at Slot{location}!")
        else:
                print("Error saving template.")
else:
        print("Fingerprint mismatch. Try again.")
#main.py
import time, serial
from RPLCD.i2c import CharLCD
import adafruit_fingerprint
lcd = CharLCD('PCF8574', 0x27, cols=16, rows=2)
uart = serial.Serial("/dev/serialo", baudrate=57600,
timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
def show(11, 12=""):
        lcd.clear()
        lcd.cursor_pos = (0, 0)
        lcd.write_string(11)
        lcd.cursor_pos = (1, 0)
        lcd.write_string(12)
show("System Ready", "Place Finger")
while True:
        if finger.get_image() == adafruit_fingerprint.OK:
                show( "Processing ... ", "Please Wait")
                if finger.image_2_tz(1) == adafruit_fingerprint.OK:
                        if finger.finger_search() == adafruit_fingerprint.OK:
                                show("RFID UID MATCHED", f"ID:{finger.finger_id}")
                                time.sleep(3)
                        else:
                                show("RFID UID","UNMATCHED")
                                time.sleep(1.5)
                                show("Access Denied", "")
                                time.sleep(2)
                show( "Place Finger", "")
        time.sleep(0.1)


#12 fingerprint sensor & relay module
sudo apt install -y python3 python3-pip python3-serial
pip3 install adafruit-circuitpython-fingerprint
sudo raspi config (serial port – hardware port yes )
#if Raspberry Pi OS restricts system-wide pip installation
python3 -m venv fingerprint-env
source fingerprint-env/bin/activate
pip install pyserial adafruit-circuitpython-fingerprint RPi.GPIO
#enroll.py
import serial
import time
import adafruit_fingerprint
uart = serial.Serial(
"/dev/serial0",
baudrate=57600,
timeout=1
)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
finger_id = int(input("Enter fingerprint ID (1-127): "))
print("Place finger on sensor...")
while finger.get_image() != adafruit_fingerprint.OK:
time.sleep(0.1)
print("Image captured")
```

```
if finger.image_2_tz(1) != adafruit_fingerprint.OK:
print("Image conversion failed")
exit()
print("Remove finger")
time.sleep(2)
print("Place the same finger again...")
while finger.get_image() != adafruit_fingerprint.OK:
time.sleep(0.1)
if finger.image_2_tz(2) != adafruit_fingerprint.OK:
print("Image conversion failed")
exit()
if finger.create_model() != adafruit_fingerprint.OK:
print("Fingerprints did not match")
exit()
if finger.store_model(finger_id) == adafruit_fingerprint.OK:
print("Fingerprint enrolled successfully")
print("Stored ID:", finger_id)
else:
print("Failed to store fingerprint")

#main.py
import serial
import time
import RPi.GPIO as GPIO
import adafruit_fingerprint
GPIO.setwarnings(False)
# Change this GPIO if required
RELAY_PIN = 17
# Active-LOW relay
RELAY_ON = GPIO.LOW
RELAY_OFF = GPIO.HIGH
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT, initial=RELAY_OFF)
try:
uart = serial.Serial(
"/dev/serial0",
baudrate=57600,
timeout=1
)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
print("Fingerprint Door Lock System")
print("Place Finger")
while True:
result = finger.get_image()
if result == adafruit_fingerprint.OK:
print("Processing...")
# Convert image to template
if finger.image_2_tz(1) != adafruit_fingerprint.OK:
print("Fingerprint processing failed")
time.sleep(1)
continue
# Search fingerprint database
if finger.finger_search() == adafruit_fingerprint.OK:
print("Access Granted")
GPIO.output(RELAY_PIN, RELAY_ON)
print("Door Unlocked")
time.sleep(5)
GPIO.output(RELAY_PIN, RELAY_OFF)
print("Door Locked")
else:
print("Access Denied")
```

```
GPIO.output(RELAY_PIN, RELAY_OFF)
time.sleep(1)
print("Place Finger")
else:
# No finger or another sensor response
time.sleep(0.1)
except Exception as e:
print("Communication error:", e)
finally:
GPIO.output(RELAY_PIN, RELAY_OFF)
GPIO.cleanup()


#14 web dashboard for flask
from flask import Flask, render_template_string
import RPi.GPIO as GPIO

app = Flask(__name__)
RELAY = 17

# GPIO setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.output(RELAY, GPIO.HIGH)
HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>Relay Control</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body {
      font-family: Arial;
      text-align: center;
      margin-top: 70px;
    }
    .status {
      font-size: 28px;
      font-weight: bold;
    }
    button {
      font-size: 25px;
      padding: 20px 40px;
      margin: 15px;
      color: white;
      border: none;
      border-radius: 8px;
    }
    .on {
      background: green;
    }
    .off {
      background: red;
    }
  </style>
</head>
<body>
  <h1>Raspberry Pi Relay</h1>

  {% if state %}
    <div class="status" style="color:green;">
```

```
      Status: ON
    </div>
  {% else %}
    <div class="status" style="color:red;">
      Status: OFF
    </div>
  {% endif %}
  <a href="/on">
    <button class="on">ON</button>
  </a>
  <a href="/off">
    <button class="off">OFF</button>
  </a>
</body>
</html>
"""
@app.route("/")
def home():
  state = GPIO.input(RELAY) == GPIO.LOW
  return render_template_string(HTML, state=state)
@app.route("/on")
def turn_on():
  GPIO.output(RELAY, GPIO.LOW)
  return render_template_string(HTML, state=True)
@app.route("/off")
def turn_off():
  GPIO.output(RELAY, GPIO.HIGH)
  return render_template_string(HTML, state=False)
try:
  app.run(host="0.0.0.0", port=5000)
finally:
  GPIO.cleanup()


#18 read cpu,memory and temp
sudo apt install python3-psutil
import psutil
import time
def get_cpu_temperature():
  try:
    with open("/sys/class/thermal/thermal_zone0/temp", "r") as file:
      temp = int(file.read()) / 1000.0
    return temp
  except FileNotFoundError:
    return None
while True:
  cpu_usage = psutil.cpu_percent(interval=1)
  memory_usage = psutil.virtual_memory().percent
  cpu_temp = get_cpu_temperature()
  print("-------------------------------------")
  print(f"CPU Usage      : {cpu_usage:.1f}%")
  print(f"Memory Usage   : {memory_usage:.1f}%")
  if cpu_temp is not None:
    print(f"CPU Temperature : {cpu_temp:.1f} °C")
  else:
    print("CPU Temperature : Not available")

  print("-------------------------------------")
  time.sleep(1)
```

#19 RFID,fingerprint sensor , lcd, relay module

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3-pip python3-serial python3-rpi.gpio i2c-tools
pip3 install --break-system-packages RPLCD
pip3 install --break-system-packages adafruit-circuitpython-fingerprint
pip3 install --break-system-packages mfrc522
sudo raspi-config
sudo i2cdetect -y 1
#rfid_test.py
from mfrc522 import SimpleMFRC522
import RPi.GPIO as GPIO
reader = SimpleMFRC522()
try:
print("Place RFID card...")
while True:
uid, text = reader.read()
print("RFID UID:", uid)
print("Data:", text)
break
finally:
GPIO.cleanup()
#enroll.py
import serial
import adafruit_fingerprint
# UART configuration on Raspberry Pi GPIO 14/15
uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)

finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
location = int(input("Enter fingerprint ID (1-127): "))
print("Place finger...")
while finger.get_image() != adafruit_fingerprint.OK:
pass
if finger.image_2_tz(1) != adafruit_fingerprint.OK:
print("Image conversion failed")
exit()
print("Remove finger")
input("Press Enter after removing finger...")
print("Place same finger again...")
while finger.get_image() != adafruit_fingerprint.OK:
pass
if finger.image_2_tz(2) != adafruit_fingerprint.OK:
print("Image conversion failed")
exit()
if finger.create_model() != adafruit_fingerprint.OK:
print("Fingerprint mismatch")
exit()
if finger.store_model(location) == adafruit_fingerprint.OK:
print("Fingerprint enrolled successfully")
else:
print("Enrollment failed")
#main.py
import RPi.GPIO as GPIO
import time
import csv
import os
import serial
from datetime import datetime
from RPLCD.i2c import CharLCD
from mfrc522 import SimpleMFRC522
import adafruit_fingerprint
```

```
# Pin Definitions (BCM numbering)
RELAY = 23
LED = 21
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)
# Active-LOW Relay: default HIGH (OFF)
GPIO.output(RELAY, GPIO.HIGH)
GPIO.output(LED, GPIO.LOW)
# Initialize I2C LCD (change 0x27 to 0x3F if i2cdetect reported 0x3F)
lcd = CharLCD(
i2c_expander='PCF8574',
address=0x27,
port=1,

cols=16,
rows=2,
charmap='A00',
auto_linebreaks=True
)
reader = SimpleMFRC522()
# Replace with the decimal UID obtained from rfid_test.py
WHITELIST = [
123456789,
987654321
]
# Initialize Fingerprint Sensor
uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
LOG_FILE = "access_log.csv"
def log_event(method, result):
new_file = not os.path.exists(LOG_FILE)
with open(LOG_FILE, "a", newline="") as file:
writer = csv.writer(file)
if new_file:
writer.writerow(["Time", "Method", "Result"])
writer.writerow([
datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
method,
result
])
def show_message(line1, line2=""):
lcd.clear()
lcd.write_string(line1[:16])
if line2:
lcd.cursor_pos = (1, 0)
lcd.write_string(line2[:16])
def unlock(method):
show_message("ACCESS GRANTED", method)
GPIO.output(LED, GPIO.HIGH)
GPIO.output(RELAY, GPIO.LOW) # Active-LOW trigger
log_event(method, "GRANTED")
time.sleep(5)
GPIO.output(RELAY, GPIO.HIGH) # Turn off relay
GPIO.output(LED, GPIO.LOW)
def deny(method):
show_message("ACCESS DENIED", method)
GPIO.output(LED, GPIO.LOW)
log_event(method, "DENIED")
time.sleep(2)
def check_fingerprint():
```

```
# Poll sensor for up to 3 seconds
start = time.time()
while time.time() - start < 3:
result = finger.get_image()
if result == adafruit_fingerprint.OK:
break
elif result == adafruit_fingerprint.NOFINGER:

time.sleep(0.1)
continue
else:
return False
else:
return False
show_message("Processing...")
if finger.image_2_tz(1) != adafruit_fingerprint.OK:
return False
if finger.finger_search() != adafruit_fingerprint.OK:
return False
return True
try:
show_message("ACCESS CONTROL", "SYSTEM READY")
time.sleep(2)
while True:
show_message("Scan RFID", "or Fingerprint")
# Non-blocking RFID check
card_id, text = reader.read_no_block()
if card_id:
print("RFID UID:", card_id)
if card_id in WHITELIST:
unlock("RFID")
else:
deny("RFID")
time.sleep(1)
continue
# Check Fingerprint sensor
if check_fingerprint():
print("Fingerprint matched")
unlock("FINGERPRINT")
time.sleep(1)
except KeyboardInterrupt:
print("\nProgram stopped")
finally:
GPIO.output(RELAY, GPIO.HIGH)
GPIO.output(LED, GPIO.LOW)
GPIO.cleanup()
lcd.clear()
```